Sentiment Analysis on Amazon Product Reviews

LSTM Baseline — CS6120 NLP Final Project (Team 102)

Xuyang Shen

March 23, 2026

Contents

1 Data Preprocessing & Embeddings **1**

2 Model Architecture **1**

3 Training Process **2**

4 Evaluation and Results **2**

4.1 Class Imbalance Analysis 2

4.2 Discussion 3

---

1 Data Preprocessing & Embeddings

The raw Amazon Software reviews were preprocessed via the following pipeline:

1. **Lowercasing & punctuation removal:** All text was converted to lowercase and stripped of punctuation using Python's `string.punctuation`.
2. **Stopword filtering:** NLTK's English stopword corpus was used to remove non-informative tokens, reducing noise and improving feature saliency.
3. **Whitespace tokenization:** Remaining tokens were split on whitespace.

Star ratings were converted to binary sentiment labels: ratings of 4–5 were labelled **Positive** (1), ratings of 1–2 were labelled **Negative** (0), and 3-star reviews were excluded. The dataset was split into Train / Validation / Test at an 80 / 10 / 10 ratio, yielding 8,964 / 1,121 / 1,121 samples.

GloVe Embeddings

Pre-trained GloVe embeddings (300-dimensional, 6B corpus) were used to build a fixed embedding matrix. A vocabulary of **16,199 words** was constructed from the training set (minimum frequency ≥ 2), and each word was mapped to its corresponding GloVe vector. Out-of-vocabulary (OOV) tokens were initialized with a random normal distribution ($\sigma = 0.6$).

2 Model Architecture

The model follows a standard sequence classification architecture:

Table 1: LSTM Model Architecture

Layer	**Configuration**	**Notes**
Embedding	16,199 $\times$ 300	GloVe weights, **frozen** (`requires_grad=False`)
LSTM (2 layers)	hidden dim = 256	`batch_first=True`; dropout = 0.5 between layers
Dropout	$p = 0.5$	Applied to final hidden state
Linear	$256 \to 1$	Fully connected classification head
Sigmoid	—	Outputs probability $\in (0, 1)$

The embedding layer is frozen to prevent overfitting on the 10K-sample subset. The final hidden state of the last LSTM layer ($\mathbf{h}_T^{(2)}$) is passed through a dropout layer and then a linear layer, yielding a scalar probability for binary classification.

3 Training Process

Table 2: Training Hyperparameters

Hyperparameter	**Value**
Optimizer	Adam
Learning rate	1×10^{-3}
Loss function	Binary Cross Entropy (`BCELoss`)
Batch size	64
Max epochs	10
Early stopping patience	3 (monitoring validation loss)
Hardware	Apple MPS (Mac GPU acceleration)

Early stopping monitored validation loss with a patience of 3. The best validation loss (≈ 0.497) was achieved at **Epoch 4**, and training was halted at **Epoch 7** as performance continued to deteriorate. The optimal model weights were restored prior to evaluation.

4 Evaluation and Results

The model was evaluated on the 10% held-out test set (1,121 reviews).

Table 3: Test Set Performance Metrics

Metric	**Value**
Accuracy	80.37%
Precision	80.50%
Recall	99.67%
F1-Score	0.8907

4.1 Class Imbalance Analysis

The confusion matrix reveals a severe class imbalance in the test set. Of the 1,121 samples, 899 (80.2%) are positive and only 222 (19.8%) are negative. This imbalance propagates from the raw Amazon review distribution, where high-star ratings dominate in the Software category.

As a result, the model learned a heavily positive-biased decision boundary:

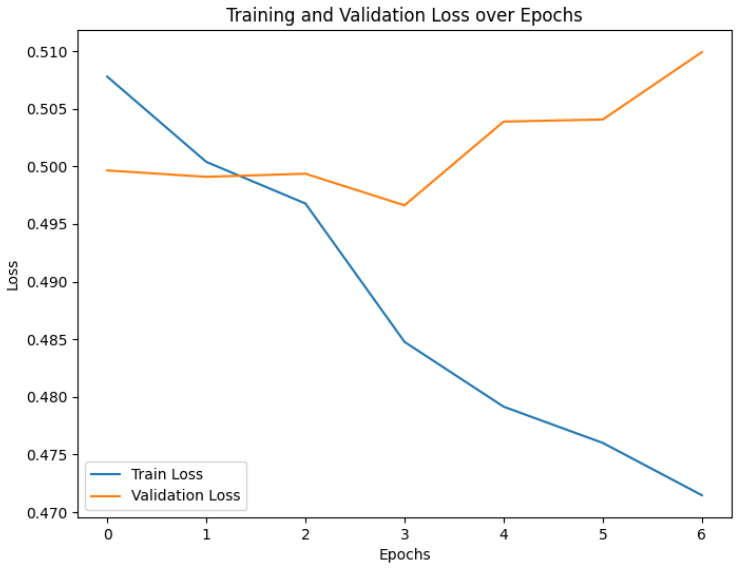

Figure 1: Training and Validation Loss over Epochs

Table 4: Confusion Matrix (Predicted vs. Actual)

		Predicted	
		Negative (0)	Positive (1)
2***Actual**	Negative (0)	**5**	217
	Positive (1)	3	**896**

- **Positive class recall:** 896 / 899 = 99.67%
- **Negative class recall:** 5 / 222 = 2.25%

The model classifies nearly all inputs as positive, effectively failing to detect negative sentiment.

4.2 Discussion

The 80.37% accuracy appears acceptable at first glance, but is largely explained by the class prior: a trivial classifier that always predicts positive would achieve ≈80.2% accuracy. The more informative metric is the near-zero true negative rate, which indicates the model has not meaningfully learned to distinguish negative sentiment.

This performance gap relative to the expected ∼85% can be attributed to two factors:

1. **Unaddressed class imbalance:** Standard `BCELoss` weights all samples equally, so the model minimizes loss by favoring the majority (positive) class.
2. **Static embeddings:** Frozen GloVe vectors cannot adapt to the nuanced, often sarcastic language patterns prevalent in negative software reviews.

A natural mitigation would be to apply a weighted loss function via the `pos_weight` parameter in `BCEWithLogitsLoss`, penalizing false negatives more heavily. This limitation provides a clear motivation for evaluating BERT in the companion section: BERT's dynamic contextual attention mechanism may better handle the linguistic complexity of minority-class reviews.